

- Considere os seguintes problemas de otimização não linear:

1. (P1) minimizar $F(w) = w_1 + w_2$
 $w \in \mathbb{R}^2$
 sujeito a
 $w_1^2 + w_2^2 - 2 = 0$; com ponto inicial $w^{(0)} = (0, 0)$.
2. (P2) minimizar $F(w) = (w_1 + 3w_2 + w_3)^2 + 4(w_1 - w_2)^2$
 $w \in \mathbb{R}^3$
 sujeito a
 $6w_2 + 4w_3 - w_1^3 - 3 \geq 0$
 $1 - w_1 - w_2 - w_3 = 0$ com ponto inicial $w^{(0)} = (0, 0)$.
 $w_1, w_2, w_3 \geq 0$;
3. (HS6) minimizar $F(w) = (1 - w_1)^2$
 $w \in \mathbb{R}^2$
 sujeito a
 $10(w_2 - w_1^2) = 0$; com ponto inicial $w^{(0)} = (-1.2, 1)$.
4. (HS32) minimizar $F(w) = \frac{1}{2}w_1^2 + w_2^2 - w_1w_2 - 2w_1 - 6w_2$
 $w \in \mathbb{R}^2$
 sujeito a
 $-w_1 - w_2 + 2 \geq 0$
 $w_1 - 2w_2 + 2 \geq 0$
 $-2w_1 - w_2 + 3 \geq 0$ com ponto inicial $w^{(0)} = (0.1, 0.7, 0.2)$.
 $w_1, w_2 \geq 0$;

1. Minimize a função de penalidade quadrática Q do problema (P1), para $\mu_k = 1, 10, 100, 1000$ usando um algoritmo de otimização sem restrições.
2. Implemente o Método de Penalidade Quadrática (Algoritmo 8.1) com $\mu_0 = a$ (para $a = 1; 10$). Faça $\mu_{k+1} = b\mu_k$ (para $b = 2; 10$), $\tau_k = 1/\mu_k$ e escolha para ponto inicial $w_s^{(k+1)}$ a solução de $Q(w; \mu_k)$.

- (a) Para critério de paragem final use as condições:

$$\|\nabla Q(w^{(k)})\|_\infty \leq \varepsilon_1 \quad \text{e}$$

$$\|c_{\mathcal{E}}(w^{(k)})\|_\infty \leq \varepsilon_2 \quad \text{e} \quad \|\max(0, -c_{\mathcal{I}}(w^{(k)}))\|_\infty \leq \varepsilon_2 \quad (\text{medida da violação})$$

com $\varepsilon_1 = \varepsilon_2 = 0^{-6}$.

- (b) Para cada problema, em cada iteração, guarde a seguinte informação sobre a resolução da penalidade quadrática $Q(w; \mu_k)$:

k	$w^{(k)}$	$\ \nabla Q(w^{(k)})\ _\infty$	$Viol(w^{(k)})$	$F(w^{(k)})$	$Q(w^{(k)})$	τ_k	μ_k
...	...	...	...	...	...	...	...

onde $Viol(w^{(k)}) = \|[c_{\mathcal{E}}(w^{(k)}); \max(0, -c_{\mathcal{I}}(w^{(k)}))]\|_\infty$.

- (c) Minimize a função de penalidade quadrática $Q(w; \mu_k)$ com a função **fminunc** do MATLAB, sem fornecer as derivadas desta função.
- (d) Teste o algoritmo com os problemas de otimização não linear com restrições fornecidos.
- (e) Analise e comente os resultados obtidos, e indique se foi encontrada uma solução (aproximada) para cada um dos problemas com restrições.

3. Resolva os problemas fornecidos utilizando a função **fmincon** do MATLAB, sem fornecer as derivadas da função objetivo e das funções de restrições não lineares. Para cada caso, registre toda a informação devolvida pela função **fmincon**. Compare os resultados com os obtidos no Exercício 2.
4. Implemente o Método de Penalidade Exata ℓ_1 (Algoritmo 8.2) com $\mu_0 = a$ (para $a = 1; 10$). Faça $\mu_{k+1} = b\mu_k$ (para $b = 2; 10$), $\tau = 10^{-6}$ e escolha para ponto inicial $w_s^{(k+1)}$ a solução de $\phi_1(w; \mu_k)$.
- (a) Para cada problema, em cada iteração, guarde a seguinte informação sobre a resolução da penalidade exata $\phi_1(w; \mu_k)$:

k	$w^{(k)}$	$\ \nabla \phi_1(w^{(k)})\ _\infty$	$Viol(w^{(k)})$	$F(w^{(k)})$	$\phi_1(w^{(k)})$	μ_k
...	...	...	...	...	...	...

onde $Viol(w^{(k)}) = \|[c_{\mathcal{E}}(w^{(k)}); \max(0, -c_{\mathcal{I}}(w^{(k)}))]\|_\infty$.

- (b) Minimize a função de penalidade ℓ_1 , $\phi_1(w; \mu_k)$ com a função **fminsearch** do MATLAB.
- (c) Teste o algoritmo com os problemas de otimização não linear com restrições fornecidos.
- (d) Analise e comente os resultados obtidos, e indique se foi encontrada uma solução (aproximada) para cada um dos problemas com restrições.
- (e) Resolva o Exercício anterior considerando agora a função **fminunc** do MATLAB para minimizar a função de penalidade ℓ_1 , $\phi_1(w; \mu_k)$. Analise e comente os resultados obtidos.

Notas: Na linha de comando do MATLAB, executar:

- `help fmincon` $\Rightarrow$ abre a [Documentation for fmincon](#);
- `fmincon` - determina o mínimo de uma função multivariável com restrições.

Sintaxe:

```
x = fmincon(fun,x0,A,b)
x = fmincon(fun,x0,A,b,Aeq,beq)
x = fmincon(fun,x0,A,b,Aeq,beq,lb,ub)
x = fmincon(fun,x0,A,b,Aeq,beq,lb,ub,nonlcon)
x = fmincon(fun,x0,A,b,Aeq,beq,lb,ub,nonlcon,options)
x = fmincon(problem)
[x,fval] = fmincon(...)
[x,fval,exitflag,output] = fmincon(...)
[x,fval,exitflag,output,lambd,grad,hessian] = fmincon(...)
```

- Calcula um minimizante de um problema com o seguinte formato:

$$\min_x f(x) \quad \text{sujeito a} \quad \begin{cases} c(x) \leq 0, \\ c_{eq}(x) = 0, \\ A \cdot x \leq b, \\ A_{eq} \cdot x = b_{eq}, \\ \ell_b \leq x \leq u_b, \end{cases}$$

onde

- b e b_{eq} são vetores dos termos independentes das restrições lineares de desigualdade e igualdade, respetivamente;
- A e A_{eq} são matrizes dos coeficientes das restrições de desigualdade e igualdade, respetivamente;
- $c(x)$ e $c_{eq}(x)$ são os vetores com as funções de restrição não lineares de desigualdade e igualdade, respetivamente;

- ℓ_b e u_b são os vetores com os limites inferiores e superiores para os valores das variáveis (designadas por restrições de limites simples);
 - $f(x)$ é uma função objetivo que devolve um escalar.
- `optimset('fmincon')` - mostra as opções predefinidas do **fmincon**